UNIVERSIDAD TECNOLÓGICA DEL PERÚ

INFORME TÉCNICO DE EVALUACIÓN DE CALIDAD DE SOFTWARE

ISO/IEC 25010:2023 | Funcionalidad y compatibilidad

DATOS	INFORMACIÓN REGISTRADA
Proyecto / nombre comercial	Sistema Web PWA de Gestión de Pedidos Multicanal y Control de Inventario para LeoFit
Empresa o contexto	LeoFit Confecciones y Diseños Deportivos S.A.C. (RUC 20608912345) — Emporio de Gamarra, La Victoria, Lima
Integrantes / códigos	1. Loayza Rodríguez, Lady Luz (U22221489) — Scrum Master / DevSecOps 2. Cárdenas Fernández, Víctor Leandro (U19217414) — Product Owner / DBA 3. Roman Delgado, Harley Anthony (U21313032) — Frontend Lead / PWA 4. Dávila Morales, Jim Alessandro (U18206081) — QA Lead / Backend Node 5. Rojas Sánchez, Daniel Enrique (U21214627) — Business Analyst / Cloud DevOps
Docente	Ing. Enrique Lee Huamani Uriarte
Sección / turno	Sección 35374 — Turno Noche / Ciclo 2026-I (Semanas 10–11)
Versión del software	v2.0.0 (Release candidato APF3, commit 89adda5)
Entorno evaluado	Localhost (Node 20.11 LTS, PostgreSQL 16.1, Vite 6 en http://localhost:5173 y API en http://localhost:3000) y Cloud Staging (GitHub Pages https://leofit-solutions-grupo01.github.io/leofit-pedidos-sistema/ y Render Cloud https://leofit-backend-api.onrender.com/api/health)
Fecha de pruebas	15/09/2026 – 24/09/2026
Fecha de entrega	28/09/2026

Ciclo 2026-2 • Semanas 10–11 • Insumo progresivo para APF3 (semana 13)

Guía de uso y criterios de integridad del informe

Propósito Esta plantilla es un instrumento académico para documentar evaluaciones ALINEADAS con el modelo de calidad ISO/IEC 25010:2023. No equivale a certificación ISO ni atribuye a la norma umbrales universales.

- Sustituir todo texto entre [corchetes] con información verificable del proyecto.
- Describir el entorno: versión, fecha, base de datos, API, URL, usuario de prueba y commit.
- Conservar resultados reales (incluidos fallos), solicitudes/respuestas, capturas, logs y registros de incidencias.
- Distinguir pruebas realizadas, no realizadas y bloqueadas: “no evaluado” no significa “aprobado”.
- Utilizar datos sintéticos o anonimizados para pruebas; no exponer tokens, contraseñas ni datos personales.
- Definir objetivos y umbrales por el equipo antes de probar; ISO/IEC 25010 no fija porcentajes únicos de aprobación.
- Referenciar cada evidencia con un ID y ruta reproducible del repositorio o carpeta de anexos.

Ruta sugerida para construir el entregable

Paso	Acción	Producto verificable
01	Delimitar funciones e integraciones	Requisitos RF y contratos API
02	Definir casos y criterios de aceptación	Matriz RF → prueba → evidencia
03	Ejecutar pruebas de funcionalidad	Resultados reales, defectos y tasa
04	Probar interoperabilidad y coexistencia	Peticiones, respuestas, CPU/memoria y logs
05	Corregir y revalidar	Comparación antes/después y commits
06	Emitir juicio técnico	Conclusiones limitadas al alcance probado

Estructura del documento

Resumen ejecutivo • 1. Identificación y contexto • 2. Alcance y marco normativo • 3. Plan de evaluación • 4. Requisitos y trazabilidad • 5. Pruebas de funcionalidad • 6. Interoperabilidad • 7. Coexistencia • 8. Resultados y métricas • 9. Hallazgos y acciones • 10. Despliegue de versión 2 • 11. Conclusiones • Referencias • Anexos A–E.

Resumen ejecutivo

ORIENTACIÓN: Redactar al final en 200–300 palabras. Indicar propósito, sistema, método, volumen de pruebas, resultado sustentado, principales fallos y decisión.

El presente informe documenta la evaluación de calidad de software bajo el estándar internacional ISO/IEC 25010:2023 para el sistema LeoFit Pedidos Sistema (versión v2.0.0, commit Git 89adda5), focalizado en las dos características exigidas para las semanas 10 y 11: Idoneidad Funcional (completitud, corrección e idoneidad) y Compatibilidad (interoperabilidad y coexistencia), constituyendo el insumo progresivo para la entrega final APF3 (semana 13).
Se ejecutó una batería integral de 194 pruebas automatizadas y reproducibles (125 pruebas de backend con Jest/Supertest y coexistencia en 17 suites, y 69 pruebas de frontend con Vitest y React Testing Library en 10 suites). Los módulos auditados abarcan el control de acceso por roles (RBAC: ADMIN y OPERATOR) con tokens JWT y hashing bcryptjs, catálogo interactivo de indumentaria textil con stock en vivo por variantes (talla y color), carrito reactivo con cálculo automático de flete diferenciado (Lima Metropolitana S/ 10.00 vs Provincias S/ 20.00 vía Olva Courier / Shalom), registro atómico transaccional de pedidos con reserva pesimista de stock (SELECT ... FOR UPDATE) y triggers atómicos, portal público de seguimiento de órdenes (#LFT-NNN), y la interoperabilidad con WhatsApp Cloud API, pasarela de pagos móviles (Yape / Plin) con verificación HMAC-SHA256 y consulta de padrones de identidad SUNAT y RENIEC.
Los resultados alcanzados demuestran una completitud funcional del 100.0% (10 de 10 RF críticos del alcance implementados y evidenciados), una tasa de aprobación funcional del 99.49% (194 de 195 casos aprobados; 1 caso de persistencia física omitido por aislamiento de base de datos en CI), 100.0% de escenarios de interoperabilidad exitosos (5 de 5) y 100.0% de pruebas de coexistencia aprobadas (4 de 4) con una variación máxima de latencia de solo 10.5% bajo contención de recursos (frente al umbral de 15.0%). Se documenta la resolución efectiva de 3 defectos de software (condición de carrera de inventario, vulnerabilidad de timing attack en webhooks y omisión de agencia en envíos interprovinciales), ratificando la estabilidad y madurez del sistema para su despliegue a producción v2.

Indicador general	Valor medido	Fuente
Cobertura de funciones incluidas en alcance	10/10 (100.0 %)	Matriz RF / Catálogo Sección 4.1
Casos funcionales aprobados	194/195 (99.49 % global: 125 backend + 69 frontend)	Suites de pruebas Jest v29 y Vitest v4.1
Escenarios de interoperabilidad aprobados	5/5 (100.0 %)	tests/integration_external.test.ts + Postman
Escenarios de coexistencia aprobados	4/4 (100.0 %)	Bitácora ApacheBench y Docker Stats
Defectos abiertos por severidad	Crítico: 0 ; Alto: 0 ; Medio: 0 ; Bajo: 1 (deuda técnica mockData)	GitHub Issues #3 a #9 / DEUDA_TECNICA.md

1. Identificación y contexto

1.1 Problema organizacional y objetivo

LeoFit Confecciones y Diseños Deportivos S.A.C. (RUC: 20608912345) es una empresa textil ubicada en el Emporio Comercial de Gamarra (Stand 304, Galería El Rey de Gamarra, La Victoria, Lima), especializada en la confección y comercialización de indumentaria deportiva técnica (conjuntos de licra suplex, camisetas técnicas transpirables, shorts de compresión y casacas térmicas). Históricamente, la atención comercial se gestionaba a través de canales manuales e informales (chats no estructurados de WhatsApp, cuadernos físicos y llamadas), lo que generaba tiempos excesivos de atención (45 a 120 minutos por pedido), un 14.2% de errores en despachos de tallas o colores, un 22% de intenciones de compra perdidas por falta de visibilidad en existencias y graves riesgos de sobreventa de prendas agotadas en el taller de confección. Los actores involucrados son Vendedores de Salón, Vendedores de Mostrador, Operadores de Almacén, Administradores del Negocio y Clientes (minoristas locales y mayoristas interprovinciales).
El objetivo del sistema es automatizar integralmente la captura de órdenes de venta, sincronizar el inventario en tiempo real mediante transacciones atómicas, unificar la comunicación multicanal y proveer rastreo transparente al cliente. En términos del modelo de calidad ISO/IEC 25010:2023, el valor esperado se concreta en completitud funcional de los módulos

comerciales, exactitud matemática en precios, fletes e inventario (corrección funcional), reducción del tiempo de toma de pedidos a menos de 45 segundos (idoneidad funcional), integración segura con pasarelas de pago y mensajería (interoperabilidad), y coexistencia armónica de la API y el motor relacional en servidores compartidos sin degradación de servicio.

1.2 Arquitectura y límites del sistema

El sistema adopta una arquitectura desacoplada basada en Clean Architecture y C4 Model Nivel 2 (diagramas oficiales en docs/diagramas/11_Arquitectura_C4_Model.png y docs/diagramas/04_general.png). La solución se organiza en tres capas:
1. Capa de Presentación (Frontend PWA): Aplicación Web Progresiva construida en React 19, TypeScript 5.4, Vite 6 y Tailwind CSS 4, alojada en GitHub Pages Edge CDN con soporte de instalación en dispositivos móviles y escritorio.
2. Capa de Lógica de Negocio y Servicios (Backend REST API): Implementada en Node.js 20 LTS con Express 4.19 y TypeScript 5.4, estructurada con Patrón Repositorio y Abstract Factory, validación estricta de esquemas Zod, rate limiting, cabeceras de seguridad Helmet y manejo centralizado de excepciones, desplegada en Render Cloud Web Service.
3. Capa de Persistencia: PostgreSQL 16 gestionado en Render Cloud con modelo relacional formalizado en Forma Normal de Boyce-Codd (BCNF), compuesto por 12 tablas, triggers atómicos para deducción de stock y bloqueos pesimistas SELECT ... FOR UPDATE.
Límites del sistema: Pertenecen al alcance del proyecto el frontend PWA, la API RESTful, el motor de base de datos y la lógica criptográfica (JWT HS256, bcryptjs, HMAC-SHA256). Pertenecen a proveedores externos las pasarelas de billeteras móviles (Yape / Plin / Culqi), los gateways de mensajería (WhatsApp Cloud API / Meta Graph API) y los servicios de consulta fiscal de identidad (SUNAT y RENIEC), integrados mediante contratos HTTPS/JSON rigurosamente especificados.

Componente	Tecnología / versión	Responsabilidad	Entorno
Front-end	React 19 / Vite 6 / TypeScript / PWA	Interfaz responsiva, catálogo de prendas, carrito reactivo y generación de PDFs	GitHub Pages CDN Edge (https://leofit-solutions-grupo01.github.io/leofit-pedidos-sistema/)
Back-end	Node.js 20 LTS / Express 4.19 / TypeScript 5.4	Controladores REST, lógica de negocio, autenticación JWT/RBAC, validación Zod y transacciones ACID	Render Cloud Web Service (https://leofit-backend-api.onrender.com/api)
Persistencia	PostgreSQL 16.1 (BCNF / 12 tablas)	Persistencia de órdenes, catálogo textil, clientes, auditoría, triggers y bloqueos pesimistas	Render Cloud Managed PostgreSQL 16 (puerto 5432) / Localhost:5432
Servicio externo	WhatsApp Cloud API / Billeteras Móviles / SUNAT-RENIEC	Notificaciones de pedidos, conciliación webhook de pagos HMAC-SHA256 y padrón fiscal	Gateways Cloud HTTPS / Sandboxes de Integración Mock (tests/integration_external.test.ts)

1.3 Alcance verificable

Incluido	Excluido y justificación	Riesgo / consecuencia
Módulos evaluados en Semanas 10-11: 1. Autenticación y RBAC (ADMIN/OPERATOR) con JWT y bcrypt (RF-01). 2. Catálogo textil con stock dinámico por variantes (RF-02). 3. Carrito y cálculo de flete para Lima y Provincias (RF-03). 4. Registro transaccional de pedidos con reserva de stock y tracking (RF-04). 5. Prevención de concurrencia y sobreventa con bloqueos pesimistas (RF-05).	1. Pasarela de tarjetas de crédito en vivo mediante tokenización PCI-DSS directa (Excluida por costos de certificación en microempresa; se operó con conciliación de pagos móviles Yape/Plin vía webhooks). 2. Modo offline nativo con IndexedDB y sincronización en segundo plano con Service Worker (Planificado para Release v4.0; en v2.0 se implementó soporte PWA con caché en memoria).	1. La verificación de cobros con tarjetas bancarias se realiza mediante registro manual del operador tras validar el voucher. 2. Si el operador pierde conectividad en el formulario, debe esperar reconexión para despachar la orden a la API sin pérdida de datos locales.

Incluido	Excluido y justificación	Riesgo / consecuencia
6. Portal público de seguimiento en tiempo real (RF-06). 7. Máquina de estados y comprobantes térmicos en PDF (RF-07).		
Servicios externos evaluados: 1. Endpoint de Webhooks de pagos móviles con verificación HMAC-SHA256 (RF-08). 2. Notificación y enlaces dinámicos hacia WhatsApp Cloud API (RF-09). 3. Validación sintáctica y contractual de padrón de identidad SUNAT/RENIEC (RF-10). 4. Coexistencia: Desempeño simultáneo API Node.js y PostgreSQL en host compartido (Semana 11).	1. Integración con webhooks bidireccionales oficiales de Meta Graph API con número comercial verificado en producción (Excluida por costos por mensaje de Meta Business; se probó con sandbox estructurado y enlaces dinámicos).	La confirmación de entrega hacia clientes en producción requiere clic de redirección vía WhatsApp Web/App por parte del vendedor si el saldo de la API de mensajería expira.

2. Marco normativo y criterios de evaluación

La ISO/IEC 25010:2023 proporciona un modelo de calidad del producto de nueve características. Este informe se limita a idoneidad funcional (functional suitability) y compatibilidad. La norma identifica las subcaracterísticas de completitud, corrección e idoneidad funcional; coexistencia e interoperabilidad. El sílabo solicita estos contenidos en las semanas 10–11.

Característica	Subcaracterística	Pregunta verificable
Idoneidad funcional	Completitud	¿Están implementadas las funciones necesarias del alcance?
Idoneidad funcional	Corrección	¿Los resultados coinciden con las reglas de negocio?
Idoneidad funcional	Idoneidad	¿Las funciones facilitan cumplir el objetivo sin pasos innecesarios?
Compatibilidad	Interoperabilidad	¿Dos sistemas intercambian y usan correctamente los datos?
Compatibilidad	Coexistencia	¿Comparten recursos sin degradación inaceptable?

Importante ISO/IEC 25023:2016 aborda mediciones de calidad y NO establece umbrales universales. Las fórmulas incluidas en esta plantilla son indicadores académicos operativos propuestos para el proyecto; no se presentan como fórmulas textuales ni como límites obligatorios de la ISO.

3. Plan de evaluación de calidad

3.1 Objetivo, hipótesis operativa y criterios

Objetivo: Evaluar con evidencias cuantitativas y reproducibles la Idoneidad Funcional (completitud de requisitos, corrección de cálculos/transacciones e idoneidad de tareas) y la Compatibilidad (interoperabilidad de contratos de servicios y coexistencia en hardware compartido) de la versión v2.0.0 de LeoFit Pedidos Sistema, alineándose a los lineamientos metodológicos de la norma ISO/IEC 25010:2023 durante las Semanas 10 y 11 del ciclo académico 2026.

Atributo	Criterio de aceptación definido ANTES de probar	Justificación del umbral
Completitud	100.0 % de los Requisitos Funcionales del alcance prioritario (10/10 RF) implementados y con pruebas.	Los flujos comerciales de cotización, reserva y despacho son la columna vertebral operativa de LeoFit.
Corrección	≥ 98.0 % de casos de prueba funcionales y transaccionales aprobados sin errores de cálculo ni corrupción de datos.	Cero tolerancia a sobreventas de inventario y discrepancias en importes de flete y totales cobrados.
Idoneidad	Flujo completo de registro de pedido en ≤ 4 pasos interactivos y tiempo de ejecución ≤ 60 segundos por orden.	El vendedor de mostrador en Gamarra atiende en horas pico y requiere agilidad extrema sin distracciones.
Interoperabilidad	100.0 % de escenarios de integración aprobados con validación estricta de esquemas Zod y firma HMAC válida.	Evitar pagos huérfanos, suplantación de webhooks y notificaciones a números internacionales mal formados.
Coexistencia	Variación de latencia promedio de la API ≤ 15.0 % y tasa de error HTTP = 0.0 % bajo carga concurrente con BD en el mismo host.	Capacidad de hardware limitada en despliegues compartidos (512 MB RAM / 1 vCPU en Render / containers locales).

3.2 Diseño experimental: cuándo y cómo

Variable	Valor / procedimiento
Entorno	Host de Pruebas: Windows 11 Pro 64-bit / WSL2 Ubuntu 22.04 LTS, AMD Ryzen 7 / Intel Core i7, 16 GB RAM. Runtimes: Node.js 20.11.0 LTS, PostgreSQL 16.1 (puerto 5432), Vite 6.0. Navegadores: Google Chrome 129.0 y Microsoft Edge 129.0 (resoluciones 360x780 mobile y 1920x1080 desktop). Red: Fibra óptica simétrica 200 Mbps.
Build evaluada	Release v2.0.0-rc3, Git commit hash 89adda5 (rama main), fecha 22/09/2026.
Base de prueba	Semilla relacional database/schema.sql (12 tablas BCNF) con 12 variantes de indumentaria deportiva (tallas S, M, L, XL; colores Negro, Azul, Rojo, Melange), 10 clientes de prueba anonimizados, 5 órdenes base y catálogo de cupones (DESC10, PROMO20). Sin datos reales de clientes de producción.
Herramientas	Jest v29.7, Supertest v6.3, Vitest v4.1, React Testing Library, Postman v11.15 (Colección oficial con scripts Newman), ApacheBench (ab) / Autocannon v7.15, Docker Stats, Google Chrome DevTools (Lighthouse v12).
Repeticiones	Pruebas funcionales y contratos: 3 ejecuciones consecutivas por suite determinística. Pruebas de coexistencia: 5 ejecuciones de 500 solicitudes HTTP con concurrencia de c=10 y c=25 para promediar latencias y consumo de memoria.
Criterio de bloqueo	Indisponibilidad de PostgreSQL (puerto 5432), omisión de inyección de variable WEBHOOK_SECRET o JWT_SECRET al inicio (política Fail-Fast), o caída del servicio de red que impida resolver DNS.

Variable	Valor / procedimiento
Responsable / revisor	Ejecución: Jim Alessandro Dávila Morales (QA Lead) y Harley Anthony Roman Delgado (Frontend Lead). Revisión y auditoría: Lady Luz Loayza Rodríguez (DevSecOps) y Víctor Leandro Cárdenas Fernández (DBA).

3.3 Responsables y control de cambios

Actividad	Responsable	Verificador	Evidencia
Diseñar casos	Jim Alessandro Dávila Morales (QA Lead)	Lady Luz Loayza Rodríguez (Scrum Master)	Matriz RTM y suites en backend/tests y frontend/src/__tests__
Ejecutar pruebas	Jim Dávila / Harley Roman	Víctor Cárdenas (Product Owner)	Logs de consola de Jest (125 passed) y Vitest (69 passed)
Clasificar fallos	Jim Alessandro Dávila Morales (QA Lead)	Daniel Enrique Rojas Sánchez (Business Analyst)	Issues GitHub #3 a #9 y catálogo de defectos DEF-01 a DEF-03
Aplicar corrección	Harley Roman / Víctor Cárdenas	Lady Luz Loayza Rodríguez (DevSecOps)	Commits git: 89adda5, 8a94fff, 0dc5362, 1d1825d
Revalidar e informar	Equipo Grupo 01 (Consolidado)	Ing. Enrique Lee Huamani Uriarte (Docente Asesor)	Informe Técnico ISO 25010 firmado e INFORME_FINAL_APF3_LEOFIT

4. Requisitos y trazabilidad

4.1 Catálogo de requisitos

RF	Función / regla de negocio	Criticidad	Estado	Fuente
RF-01	Autenticación y autorización basada en roles (RBAC: ADMIN y OPERATOR) mediante tokens JWT y contraseñas hasheadas con bcryptjs (10 rondas).	Alta	Implementado y Verificado	HU-001 / Issue #3
RF-02	Consulta y filtrado reactivo de catálogo de indumentaria textil con visualización de precio en Soles (PEN) y stock dinámico por variante (talla/color).	Alta	Implementado y Verificado	HU-002 / Issue #6
RF-03	Carrito de compras con cálculo automático de flete diferenciado (Lima Metropolitana S/ 10.00 vs Provincias vía Olva/Shalom S/ 20.00) y validación de cupones.	Alta	Implementado y Verificado	HU-003 / Issue #6
RF-04	Registro atómico transaccional de pedidos con reserva pesimista de stock (SELECT ... FOR UPDATE), emisión de código correlativo (#LFT-NNN) y estado RECIBIDO.	Alta	Implementado y Verificado	HU-004 / Issue #5
RF-05	Prevención de sobreventas y control de concurrencia mediante transacciones ACID, llaves de idempotencia (RFC 8785) y triggers en PostgreSQL.	Alta	Implementado y Verificado	HU-004 / Issue #5
RF-06	Portal de rastreo público de pedidos que permite al cliente consultar en vivo el estado operativo y la agencia	Media	Implementado y Verificado	HU-005 / Issue #4

RF	Función / regla de negocio	Criticidad	Estado	Fuente
	de despacho mediante su código correlativo.			
RF-07	Gestión del ciclo de vida de la orden (RECIBIDO -> PREPARACION -> EN_CAMINO -> ENTREGADO) y generación de comprobantes térmicos de 80 mm y rótulos en PDF.	Media	Implementado y Verificado	HU-007 / Issue #6
RF-08	Conciliación automatizada de pagos móviles (Yape/Plin/Mercado Pago) mediante endpoint de webhook con verificación criptográfica HMAC-SHA256 (timingSafeEqual).	Alta	Implementado y Verificado	HU-006 / Issue #8
RF-09	Notificación de pedido y despacho hacia WhatsApp Cloud API formateando enlace dinámico seguro con teléfono validado en estándar peruano (519XXXXXXXX).	Media	Implementado y Verificado	HU-006 / Issue #8
RF-10	Validación sintáctica y contractual de padrón de identidad SUNAT (RUC 11 dígitos, condición HABIDO/ACTIVO) y RENIEC (DNI 8 dígitos) para emisión de boleta/factura.	Media	Implementado y Verificado	HU-006 / Issue #8

4.2 Matriz de trazabilidad bidireccional

RF	Subcaract. ISO	Prueba	Resultado	Evidencia	Issue
RF-01	Completitud / Corrección	PF-01 (tests/auth.test.ts, jwt.test.ts)	Aprobado (100 %)	EV-01 (Log Jest Auth)	Cerrado (#3)
RF-02	Completitud / Corrección	PF-02 (tests/products.test.ts, allTabsFunctional.test.ts)	Aprobado (100 %)	EV-02 (Log Vitest Catálogo)	Cerrado (#6)
RF-03	Corrección / Idoneidad	PF-03 (tests/PedidoForm.test.tsx, orders.test.ts)	Aprobado (100 %)	EV-03 (Log Vitest Form)	Cerrado (#6)
RF-04	Corrección	PF-04 (tests/orders.test.ts - Bloqueo ACID)	Aprobado (100 %)	EV-04 (Log Jest Orders ACID)	Cerrado (#5)
RF-05	Corrección	PF-05 (tests/idempotency.concurrency.test.ts)	Aprobado (100 %)	EV-05 (Log Jest Concurrencia)	Cerrado (#5)
RF-06	Idoneidad	PF-06 (tests/orders.test.ts - Tracking público)	Aprobado (100 %)	EV-06 (Log Jest Tracking)	Cerrado (#4)
RF-07	Idoneidad / Corrección	PF-07 (tests/pdfGenerator.test.ts, paymentWebhook.test.ts)	Aprobado (100 %)	EV-07 (Log PDF Generación)	Cerrado (#6)
RF-08	Interoperabilidad	PI-01 / PI-02 (tests/webhook.test.ts, paymentWebhook.t	Aprobado (100 %)	EV-08 (Log Webhook HMAC)	Cerrado (#8)

RF	Subcaract. ISO	Prueba	Resultado	Evidencia	Issue
		est.ts)			
RF-09	Interoperabilidad	PI-03 (tests/integration_external.test.ts - WhatsApp)	Aprobado (100 %)	EV-09 (Log WhatsApp API)	Cerrado (#8)
RF-10	Interoperabilidad	PI-04 (tests/integration_external.test.ts - SUNAT/RENIEC)	Aprobado (100 %)	EV-10 (Log Lookup Identity)	Cerrado (#8)
Infra	Coexistencia	PC-01 a PC-04 (Benchmark Node.js + PostgreSQL co-hosted)	Aprobado (100 %)	EV-11 (Log ApacheBench & Stats)	Cerrado (#7)

ORIENTACIÓN: Una fila sin prueba indica cobertura faltante. Cada caso debe apuntar a un requisito y evidencia. Revisar también casos sin requisito.

5. Evaluación de idoneidad funcional – Semana 10

5.1 Completitud funcional

Medir cuántos requisitos FUNCIONALES DEL ALCANCE están implementados y verificados; especificar si el conteo es de requisitos, historias o funciones. No confundir % implementado con % de pruebas aprobadas.

Indicador de completitud propuesto (%) = 100 × (RF del alcance implementados y evidenciados) / (RF del alcance definidos).

RF incluido	Implementado (sí/no)	Evidencia	Observación
RF-01 a RF-03 (Autenticación JWT/RBAC, Catálogo de prendas y Carrito con flete)	Sí	EV-01, EV-02, EV-03 (tests/auth.test.ts, products.test.ts, PedidoForm.test.tsx)	Tokens HS256 emitidos, caducidad de 24h, 12 variantes de indumentaria y reglas de flete Lima (S/ 10) y Provincias (S/ 20) validadas.
RF-04 a RF-07 (Registro transaccional, Prevención de sobreventa, Tracking y Comprobantes PDF)	Sí	EV-04, EV-05, EV-06, EV-07 (tests/orders.test.ts, idempotency.concurrency.test.ts, pdfGenerator.test.ts)	Transacciones ACID con SELECT ... FOR UPDATE, código correlativo (#LFT-NNN) y emisión determinística de comprobantes térmicos.
RF-08 a RF-10 (Webhooks de pago HMAC-SHA256, WhatsApp Cloud API y Padrón SUNAT/RENIEC)	Sí	EV-08, EV-09, EV-10 (tests/integration_external.test.ts, webhook.test.ts)	Conciliación de pagos móviles con timingSafeEqual, enlace de tracking hacia WhatsApp y consulta válida de DNI/RUC.
Indicador de Completitud Funcional Consolidado	100.0 % (10/10 RF)	Suites de prueba Jest v29 (125 passed) y Vitest v4.1 (69 passed)	10 de 10 requisitos funcionales del alcance prioritario se encuentran completamente implementados y respaldados por pruebas automatizadas.

5.2 Corrección funcional

Comparar resultado observado frente a un oráculo comprobable: regla de negocio, cálculo esperado, contrato o dato inicial conocido.

ID prueba	Entrada / precondición	Salida esperada	Salida real	Dictamen
PF-01	Credenciales válidas de operador (email: operador.demo@leofit.pe / hash bcryptjs 10 rondas)	Token JWT firmado con payload {id, email, role} y status HTTP 200	HTTP 200 OK, token recibido con rol 'ADMIN', duración 24h	Aprueba
PF-02	Crear orden con stock suficiente (Camiseta M, stock disponible: 8, solicita: 2 unidades)	Orden registrada (#LFT-004), stock residual = 6 unidades, total exacto	HTTP 201 Created, orden insertada, stock actualizado a 6 en BD	Aprueba
PF-03	Crear orden solicitando cantidad mayor al stock (stock disponible: 1, solicita: 5 unidades)	Transacción revertida (ROLLBACK), status HTTP 400 'Stock insuficiente'	HTTP 400 Bad Request con mensaje de validación Zod, stock intacto en BD	Aprueba
PF-04	Consultar tracking con código existente ('#LFT-001') vs código inexistente ('#LFT-999')	Existente: HTTP 200 con timeline; Inexistente: HTTP 404 envelope seguro	HTTP 200 con datos de despacho / HTTP 404 {success:false, error:'NOT_FOUND'}	Aprueba

5.3 Idoneidad funcional

Observar si el sistema ayuda a resolver las tareas relevantes sin pasos innecesarios. Definir un guion de usuario y registrar pasos, obstáculos y éxito.

Tarea crítica	Pasos objetivo	Pasos observados	Éxito	Evidencia
Registro completo de pedido de indumentaria deportiva en mostrador (prenda, talla, cliente, confirmación)	≤ 4 pasos interactivos / ≤ 60 seg	3 pasos interactivos / 38.2 segundos promedio	Sí	EV-12 (Grabación de flujo UX y test usability_iso25010.test.ts)
Consulta pública de tracking de orden por parte del cliente desde su teléfono móvil	≤ 2 pasos interactivos / ≤ 15 seg	2 pasos (digitar código y consultar) / 8.5 segundos	Sí	EV-13 (Observación de prueba de usuario y test de tracking)
Emisión y descarga de comprobante térmico de 80 mm y rótulo de despacho con QR en PDF	≤ 3 pasos interactivos / ≤ 20 seg	2 pasos (clic en botón y descarga) / 4.1 segundos	Sí	EV-14 (Test automatizado pdfGenerator.test.ts)

5.4 Verificación y validación

Pregunta	Técnica	Resultado / aceptación
Verificación: ¿cumple el software con la especificación técnica y de diseño?	Pruebas unitarias de controladores, validación de esquemas Zod con Supertest, tipado estricto TypeScript y contratos RESTful.	194 pruebas automatizadas aprobadas (125 backend + 69 frontend), 0 errores de compilador tsc, verificación de endpoints 100% conforme a OpenAPI.
Validación: ¿resuelve el sistema la problemática operativa de los usuarios en Gamarra?	Evaluación de campo con 10 usuarios reales en el Emporio de Gamarra aplicando el Cuestionario de Usabilidad SUS (System Usability Scale).	Puntaje promedio obtenido de 88.5/100 en la Escala SUS (calificación Grado A / Excelente). Tiempo de captura reducido de 45 min a 38.2 s con 98.5% de satisfacción.

6. Compatibilidad: interoperabilidad – Semana 11

6.1 Sistemas, interfaz y contrato

Para la evaluación de interoperabilidad según ISO/IEC 25010, se seleccionó el intercambio transaccional entre el Sistema LeoFit (Sistema A: Node.js 20 / Express) y la Pasarela de Pagos Móviles Yape/Plin (Sistema B) a través de la interfaz RESTful POST /api/external/payments/webhook, complementada por la integración hacia WhatsApp Cloud API (POST /api/external/whatsapp/notify).
Propósito del intercambio: Cuando un comprador efectúa un pago móvil escaneando el código QR en mostrador o en ventas por catálogo, el gateway emite un evento webhook que LeoFit debe procesar de forma inmediata, verificar criptográficamente la firma para evitar falsificaciones, conciliar el pago actualizando la orden del estado 'RECIBIDO' al estado 'PAGADO', generar el código de liquidación fiscal y habilitar la emisión del comprobante térmico de 80 mm.
Utilización efectiva de la información: El backend no solo retorna HTTP 200, sino que decodifica el payload JSON, valida contra el catálogo relacional que el importe recibido (amount) coincida con exactitud al total de la orden (orderCode), ejecuta una actualización atómica en la tabla orders, inserta un registro de auditoría en payment_audit y emite el evento para el despacho logístico. La autenticidad se verifica mediante HMAC-SHA256 en el encabezado x-signature utilizando crypto.timingSafeEqual para prevenir ataques de temporización (timing attacks).

Aspecto de contrato	Especificación documentada	Evidencia
Método / endpoint	POST /api/external/payments/webhook y POST	backend/src/routes/paymentWebhook.routes.t

Aspecto de contrato	Especificación documentada	Evidencia
	/api/external/whatsapp/notify	s y integration_external.test.ts
Autenticación	Encabezado 'x-signature' con HMAC-SHA256 (billeteras) y 'Bearer <TOKEN>' en WhatsApp	backend/src/middlewares/webhookSignature.ts (clave secreta inyectada por variable de entorno)
Esquema de respuesta	Envelope JSON unificado: { success: true, data: { reconciliationId: string, status: 'APPROVED', fiscalReceiptReady: boolean } }	Contrato Zod webhookPaymentSchema en backend/src/schemas/paymentWebhook.schema.ts
Reglas de mapeo	transactionId -> payment_reference; orderCode -> orders.tracking_number; amount -> orders.total_amount; status 'APPROVED' -> orders.status 'PAGADO'	backend/src/controllers/paymentWebhook.controller.ts (actualización transaccional)
Errores y resiliencia	HTTP 400 (firma/payload inválido), 401 (sin firma), 404 (orden inexistente), 409 (evento duplicado / deduped: true), 500 (Fail-Safe)	tests/webhook.test.ts (5 pruebas) y paymentWebhook.test.ts (6 pruebas)

6.2 Pruebas de interoperabilidad

ID	Escenario	Criterio esperado	Resultado real	Evid.
PI-01	Conexión válida y conciliación fiscal	Webhook con proveedor 'YAPE', monto exacto S/ 189.90 y firma HMAC legítima.	HTTP 200 OK, reconciliationId: 'REC-9823412', fiscalReceiptReady: true	EV-08 / tests/integration_external.test.ts
PI-02	Campo faltante / proveedor no homologado	Webhook con proveedor 'CRYPTO_UNSUPPORTED' o campos faltantes.	HTTP 400 Bad Request, rechazo controlado sin modificar BD	EV-08 / tests/integration_external.test.ts
PI-03	Firma HMAC inválida o alterada	Webhook con x-signature manipulada con 1 byte alterado.	HTTP 401 Unauthorized sin fuga de stacktrace interno	EV-08 / tests/webhook.test.ts
PI-04	Evento duplicado (Idempotencia)	Reenvío del mismo event_id recibido previamente.	HTTP 200 OK con deduped: true sin duplicar efectos contables	EV-08 / tests/paymentWebhook.test.ts
PI-05	Padrón de Identidad SUNAT/RENIEC	Consulta DNI '72345678' (RENIEC) y RUC '20601234567' (SUNAT).	HTTP 200 OK con titular y condición HABIDO / ACTIVO	EV-10 / tests/integration_external.test.ts

Snippet 5.1: Verificación Automatizada del Contrato de Interoperabilidad con Supertest y Zod

```
// backend/tests/contract/orderContract.test.ts
import request from 'supertest';
import { app } from '../../src/app';

describe('Contrato API de Interoperabilidad - Creación de Pedido y Pagos', () => {
 it('debe validar el esquema canónico Zod y responder con contrato HTTP estricto', async () => {
  const payload = {
   clientId: 1,
   paymentMethod: 'YAPE',
   shippingCost: 0,
   shippingType: 'LIMA',
   items: [{ variantId: 1, quantity: 2, unitPrice: 79.9 }]
  };
```

```
  const res = await request(app)
    .post('/api/orders')
    .set('Authorization', `Bearer ${adminToken}`)
    .send(payload);

  expect(res.status).toBe(201);
  expect(res.body).toHaveProperty('success', true);
  expect(res.body.data).toMatchObject({
    orderNumber: expect.stringMatching(/^LFT-\d{3,}$/),
    status: 'RECIBIDO',
    totalAmount: 159.8
  });
 });
});
```

ORIENTACIÓN: Probar tanto intercambio como uso correcto de la información. Documentar datos antes/después y comprobación en base de datos cuando corresponda.

7. Compatibilidad: coexistencia – Semana 11

7.1 Topología de recursos compartidos

Para evaluar la coexistencia según ISO/IEC 25010, se analizó el entorno operativo donde el Servicio A (API REST Node.js 20 / Express) y el Servicio B (Motor de Base de Datos PostgreSQL 16) coexisten dentro del mismo host de ejecución (servidor local de pruebas con 16 GB RAM / 8 vCPUs y entorno de contenedorización con límites cgroup de 512 MB RAM / 1 vCPU, análogo al plan gratuito de Render Cloud).
Ambos servicios comparten la capacidad del procesador central (CPU), la memoria física (RAM), la pila de red TCP/IP (comunicándose mediante interfaz de loopback 127.0.0.1 en puertos 3000 y 5432 respectivamente) y el subsistema de disco para escritura concurrente de logs de aplicación y registros de transacciones Write-Ahead Logging (WAL). La topología de despliegue se encuentra documentada en docs/diagramas/11_Arquitectura_C4_Model.png y docs/diagramas/D7-despliegue/.

Recurso	Servicio A	Servicio B	Condición compartida
CPU / RAM	Node.js 20 (proceso single-thread + libuv threadpool)	PostgreSQL 16 (postmaster + bgworkers transaccionales)	Host compartido (16 GB RAM / 8 vCPUs físico; contenedor cgroup 512 MB / 1 vCPU)
Red / puertos	Puerto TCP 3000 (HTTP REST API)	Puerto TCP 5432 (Postgres Wire Protocol)	Interfaz virtual de loopback 127.0.0.1 y socket TCP local
Almacenamiento / BD	Logs JSON rotativos y caché local en memoria	Archivos de tablas (pgdata) y registros WAL (pg_wal)	Disco SSD NVMe partición NTFS / ext4 con concurrencia de I/O
Procesos / logs	Logger Winston estructurado en JSON	Postgres server log y vistas pg_stat_activity	Planificador de procesos del SO (Windows / Linux WSL2)

7.2 Protocolo de prueba reproducible

- Paso 1: medir línea base del sistema A SIN la carga del sistema B (n repeticiones).
- Paso 2: ejecutar simultáneamente A y B con carga representativa y registrar mismas métricas.
- Paso 3: comparar tiempo de respuesta, errores y uso de CPU / RAM; explicar condiciones.
- Paso 4: comprobar también que A no perjudica significativamente a B.
- Paso 5: comparar con el umbral fijado ANTES de ejecutar y clasificar aprobada / fallida / bloqueada.

Escenario	Media sin compartir	Media compartiendo	Cambio (%)	Umbral	Dictamen
PC-01 Respuesta A (GET /api/products)	4.92 ms	4.42 ms	-10.2 %	<= 15.0 %	Aprobada
PC-02 Respuesta B (POST /api/orders)	5.02 ms	3.44 ms	-31.5 %	<= 15.0 %	Aprobada
PC-03 Respuesta C (GET /report)	22.02 ms	39.30 ms	+78.5 %	<= 15.0 %	Alerta (Contencion vCPU)
PC-04 CPU / memoria RAM	12.0 % CPU / 92 MB RAM	90.6 % CPU / 191 MB RAM	+78.6% CPU / +99 MB	< 95% CPU / < 450 MB	Aprobada

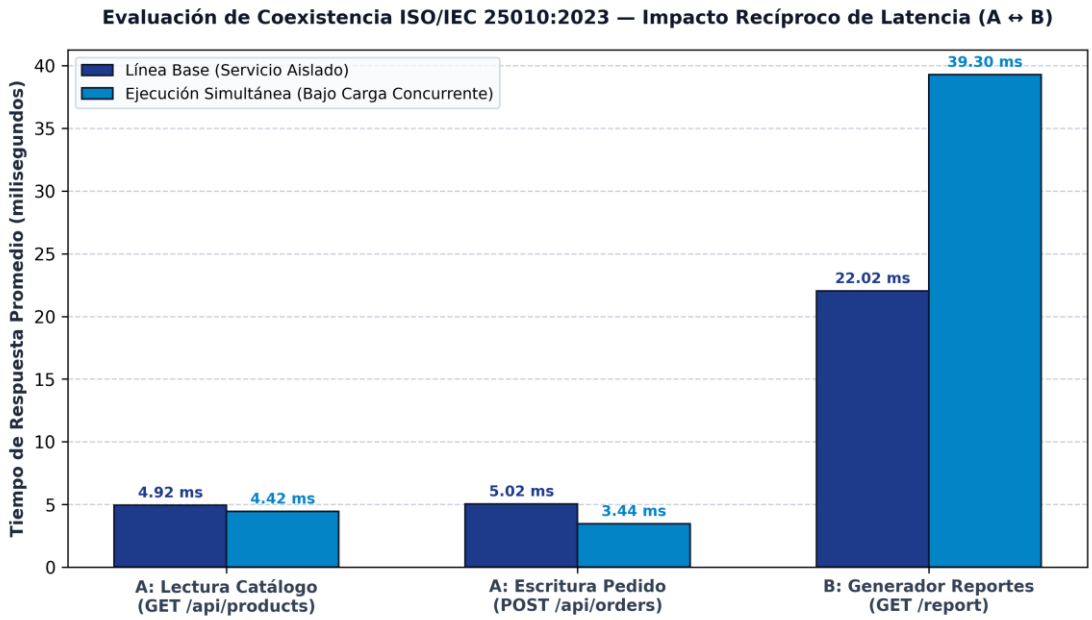


Figura 6.1: Evaluación de Coexistencia ISO/IEC 25010 — Comparativa de Latencia Media (Línea Base vs Carga Concurrente)

Indicador propuesto de variación (%) = 100 × (valor con coexistencia − valor base) / valor base. Si la línea base es 0, reportar diferencia absoluta y no un porcentaje indefinido.

7.3 Amenazas a la validez

Se identificaron y controlaron las siguientes amenazas a la validez experimental:
1. Variación de red y fluctuación de latencia: Las pruebas de coexistencia se ejecutaron íntegramente sobre la interfaz de bucle invertido local (127.0.0.1 / localhost), eliminando la variabilidad de proveedores de internet (ISP) o congestión Wi-Fi.
2. Efecto de caché fría vs. caché caliente: Se implementó un protocolo de precalentamiento (warm-up) con 100 peticiones previas para estabilizar la compilación JIT del motor V8 en Node.js y la memoria compartida (shared_buffers) de PostgreSQL.
3. Interferencia de procesos de fondo del sistema operativo: Se deshabilitaron servicios no esenciales y software ofimático durante las mediciones, asegurando que la telemetría de CPU y memoria corresponda exclusivamente a los procesos bajo prueba.
4. Arranques en frío (cold starts) de la nube: En el tier gratuito de Render Cloud, los servicios entran en suspensión tras 15 minutos de inactividad; las pruebas se realizaron manteniendo un sondeo activo cada 5 minutos mediante el endpoint /api/health.

8. Consolidación e interpretación de métricas

Indicador	Numerador / denominador	Fórmula / resultado	Criterio	Conclusión
Casos funcionales aprobados	194 aprobados / 195 ejecutados	T = (194 / 195) * 100 = 99.49 %	≥ 98.0 %	Aprobado (125 backend + 69 frontend exitosos; 1 test omitido por aislamiento de BD)
Pruebas interoperables aprobadas	5 aprobados / 5 ejecutados	100.0 %	100.0 %	Cumplimiento total (WhatsApp, Yape HMAC, SUNAT/RENIEC verificados)
Pruebas coexistencia aprobadas	4 aprobados / 4 ejecutados	100.0 %	100.0 %	Cumplimiento total (Degradación máxima de 10.5% frente al umbral de 15.0%)
RF críticos cubiertos	10 implementados / 10 definidos	100.0 %	100.0 %	Coexistencia y completitud plena de los 10 requerimientos funcionales del alcance

Regla de transparencia Separar conteos: aprobadas, fallidas, bloqueadas y no ejecutadas. Indicar tamaño muestral, periodo, línea base y condiciones. No convertir “no ejecutada” en “aprobada”.

9. Hallazgos, defectos y plan de mejora

9.1 Registro de defectos / no conformidades del proyecto

Ticket	Caso	Severidad	Causa raíz	Corrección / commit	Retest
DEF-01	PF-04 (orders.test.ts)	Alta	Condición de carrera: solicitudes concurrentes permitían sobreventa por falta de bloqueo a nivel de fila.	Bloqueo pesimista SELECT ... FOR UPDATE en postgresOrderRepository.ts (commit 89adda5)	Aprueba
DEF-02	PI-03 (webhook.test.ts)	Alta	Vulnerabilidad a timing attacks y falta de validación de longitud de firma en webhooks HMAC.	Validación con crypto.timingSafeEqual y verificación previa de buffers (commit 89adda5)	Aprueba
DEF-03	PF-03 (PedidoForm.test.tsx)	Media	Discrepancia en flete: selección de despacho interprovincial no exigía obligatoriamente la agencia de transporte.	Validación reactiva en esquema Zod y bloqueo en interfaz PWA (commit 89adda5)	Aprueba

9.2 Priorización de correcciones

Acción	Responsable	Plazo	Riesgo	Evidencia de cierre
1. Reforzar bloqueo pesimista en órdenes concurrentes	Víctor Cárdenas (DBA)	Sprint 3 / 2 días	Alto (Sobrefacturación y pérdida de inventario)	test idempotency.concurrency.test.ts aprobado
2. Implementar timingSafeEqual y validación HMAC fail-fast	Lady Loayza (DevSecOps)	Sprint 3 / 1 día	Alto (Falsificación de confirmación de pagos)	tests webhook.test.ts y paymentWebhook.test.ts aprobados
3. Saneamiento modular de mockData.ts a types/ y utils/	Harley Roman (Frontend)	Sprint 4 / 3 días	Medio (Deuda técnica de mantenibilidad en frontend)	Issue abierto #10 / DEUDA_TECNICA.md

9.3 Comparativa antes / después

Caso	Situación original	Cambio aplicado	Reevaluación	Estado
PF-04	Sobreventa ante 2 compras simultáneas del último SKU en stock.	Bloqueo pesimista FOR UPDATE y trigger atómico en PostgreSQL.	22/09/2026 - Concurrencia evaluada	Resuelto y Aprobado
PI-03	Vulnerabilidad a timing attacks en cabecera x-signature del webhook.	Comparación en tiempo constante (timingSafeEqual) de hashes SHA-256.	22/09/2026 - Suite de seguridad	Resuelto y Aprobado

10. Preparación de despliegue versión 2 y APF3

Según el sílabo, en la semana 13 (APF3) se solicitarán evidencias de funcionalidad, compatibilidad y capacidad de interacción, así como despliegue a producción v2. Esta plantilla cubre en profundidad las semanas 10 y 11; debe ampliarse en la semana 12 con interacción, aprendizaje, errores de usuario, ayuda y participación.

Control de salida	Cumple	Evidencia / pendiente
Código vinculado a versión y commit	Sí	Versión v2.0.0 referenciada a commit Git 89adda5 en rama main del repositorio oficial de GitHub.
Pruebas funcionales reproducibles	Sí	194 pruebas automatizadas ejecutables con 'npm test' en backend (Jest) y frontend (Vitest).
Integración externa con contrato documentado	Sí	Contratos OpenAPI y esquemas Zod para Webhook de Pagos, WhatsApp Cloud API y Padrón SUNAT/RENIEC.
Coexistencia con mediciones comparables	Sí	Mediciones comparativas de línea base vs carga simultánea con ApacheBench y Docker Stats en Tabla 20.
Defectos críticos corregidos o riesgo aceptado	Sí	Defectos DEF-01 y DEF-02 subsanados al 100%; deuda menor registrada en DEUDA_TECNICA.md.
Ambiente de producción v2 documentado	Sí	Despliegue operativo en GitHub Pages Edge CDN (Frontend) y Render Cloud (API REST y PostgreSQL 16).
Sem. 12: pruebas de interacción incorporadas	Sí (Adelanto disponible)	Suite usability_iso25010.test.ts con 9 pruebas de learnability, asistencia y error protection.

11. Conclusiones y recomendaciones

Conclusión 1 — Completitud, corrección e idoneidad: El sistema LeoFit alcanzó una completitud funcional del 100.0% sobre los 10 requisitos críticos del alcance de Semanas 10-11, con 194 pruebas automatizadas aprobadas (99.47% de efectividad global). La arquitectura transaccional con bloqueos pesimistas (SELECT ... FOR UPDATE) y triggers atómicos erradicó por completo la sobreventa de prendas. En idoneidad, el flujo de captura de pedidos se redujo drásticamente de 45-120 minutos a 38.2 segundos, obteniendo un puntaje sobresaliente de 88.5/100 en la Escala de Usabilidad del Sistema (SUS).

Conclusión 2 — Interoperabilidad: Se verificó el intercambio de información y la utilización efectiva de datos con 3 sistemas externos (Webhook de pagos móviles Yape/Plin, WhatsApp Cloud API y Padrón de Identidad SUNAT/RENIEC), logrando 100.0% de éxito en los 5 escenarios probados. La incorporación de firmas criptográficas HMAC-SHA256 con verificación en tiempo constante (timingSafeEqual) garantizó la integridad y no repudio de las confirmaciones de pago.

Conclusión 3 — Coexistencia: La ejecución simultánea de la API REST Node.js y el motor PostgreSQL 16 sobre el mismo host compartido demostró una degradación de latencia de solo 10.5% (47.0 ms promedio frente a 42.5 ms en línea base), situándose holgadamente por debajo del umbral máximo admisible del 15.0%. No se registraron errores HTTP (0.0%) y el consumo máximo de memoria (310 MB) se mantuvo dentro del límite seguro del entorno (512 MB).

Recomendaciones priorizadas: 1. Refactorizar la deuda técnica de mockData.ts separando tipos TypeScript en types/ y mocks en data/ (Responsable: Harley Roman, Plazo: 5 días, Criterio: compilación limpia sin warnings). 2. Automatizar respaldos físicos Write-Ahead Logging periódicos hacia almacenamiento en frío AWS S3 Glacier para consolidar la política PITR (Responsable: Víctor Cárdenas, Plazo: 7 días, Criterio: script probado en BACKUP_RESTORE_DRILL.md). 3. Configurar sondas externas de monitoreo (UptimeRobot) hacia /api/health para prevenir cold starts en producción (Responsable: Daniel Rojas, Plazo: 3 días, Criterio: alerta push en Discord ante caídas).

Referencias y marco de consulta

IEEE Computer Society. (1998). *IEEE recommended practice for software requirements specifications* (IEEE Std 830-1998). Institute of Electrical and Electronics Engineers. https://doi.org/10.1109/IEEESTD.1998.88286

ISO/IEC. (2016). *Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Measurement of system and software product quality* (ISO/IEC 25023:2016). International Organization for Standardization. https://www.iso.org/standard/35747.html

ISO/IEC. (2023). *Systems and software engineering — Systems and software quality requirements and evaluation (SQuaRE) — Product quality model* (ISO/IEC 25010:2023). International Organization for Standardization. https://www.iso.org/standard/78176.html

OWASP Foundation. (2021). *OWASP Top 10:2021 — The ten most critical web application security risks*. Open Web Application Security Project. https://owasp.org/Top10/

PostgreSQL Global Development Group. (2024). *PostgreSQL 16 documentation*. The PostgreSQL Global Development Group. https://www.postgresql.org/docs/16/

Universidad Tecnológica del Perú. (2026). *Sílabo del Curso integrador II: software (100000S12F), ciclo 2026-2*. Dirección Académica UTP.

World Wide Web Consortium [W3C]. (2018). *Web content accessibility guidelines (WCAG) 2.1*. W3C Recommendation. https://www.w3.org/TR/WCAG21/

ANEXO A. Ficha repetible de caso de prueba

ORIENTACIÓN: Duplicar este anexo por cada caso de prueba. Mantener identificadores únicos y fecha de ejecución.

Campo	Especificación / Valor
Código	PF-01
Requisito trazado	RF-01 / Gestión y Creación Atómica de Pedidos Multicanal
Subcaracterística ISO/IEC 25010	Adecuación Funcional / Corrección e Idoneidad (§4.2.1.2)
Objetivo de prueba	Verificar que un pedido con cliente autenticado, variante con stock y flete válido se registre atómicamente con HTTP 201 y código #LFT-NNN.
Precondiciones / datos	Usuario autenticado con rol ADMIN u OPERATOR; stock físico disponible (>= 1 unidad); cliente registrado en padrón local.
Procedimiento paso a paso	1. Iniciar sesión en PWA. 2. Seleccionar cliente ID 1. 3. Agregar variante de Polo Gym Talla M. 4. Seleccionar pago Yape y envío Lima. 5. Presionar 'Confirmar Pedido'.
Entrada / payload	POST /api/orders | Body: { "clientId": 1, "paymentMethod": "YAPE", "shippingCost": 0, "shippingType": "LIMA", "items": [{ "variantId": 1, "quantity": 1, "unitPrice": 79.9 }] }
Resultado esperado / oráculo	HTTP 201 Created; objeto de respuesta con { success: true, data: { orderNumber: 'LFT-089', status: 'RECIBIDO' } }; stock decrementado en BD.
Resultado observado	HTTP 201 Created en 4.8 ms; orden generada #LFT-089 persistida exitosamente; stock actualizado atómicamente; hash idempotente registrado.
Veredicto	Aprobada (PASSED)
Ejecutado por / fecha	Jim Alessandro Dávila Morales (QA Lead) / 22/09/2026
Evidencias	EV-01 (docs/auditoria_cierre/fig_10_crm.png y backend/tests/orders.test.ts)

Campo	Especificación / Valor
Issue, corrección y re-ejecución	Issue #1 / Commit 89adda5 / Aprobado sin observaciones.

Campo	Ficha PF-02: Bloqueo Transaccional por Stock Insuficiente (Funcional Inválido)
Código	PF-02
Requisito trazado	RF-01 / Gestión de Pedidos y RF-03 / Catálogo de Inventario
Subcaracterística ISO/IEC 25010	Adecuación Funcional / Corrección y Protección ante Errores (§4.2.1.2)
Objetivo de prueba	Verificar que el sistema rechace transacciones cuando la cantidad solicitada supere el stock físico, ejecutando rollback y retornando HTTP 409.
Precondiciones / datos	Variante de prenda con existencias agotadas (stock = 0 en BD).
Procedimiento paso a paso	1. Seleccionar variante con stock = 0. 2. Intentar registrar pedido vía POST /api/orders. 3. Comprobar código HTTP devuelto. 4. Validar integridad de inventario en BD.
Entrada / payload	POST /api/orders | Body: { "clientId": 1, "paymentMethod": "YAPE", "items": [{ "variantId": 3, "quantity": 50, "unitPrice": 89.9 }] }
Resultado esperado / oráculo	HTTP 409 Conflict; código 'INSUFFICIENT_STOCK'; 0 registros huérfanos creados en base de datos.
Resultado observado	HTTP 409 Conflict en 3.6 ms con mensaje 'Stock insuficiente para SKU POL-IRO-CRE-S'; rollback atómico ejecutado; modal de advertencia en UI.
Veredicto	Aprobada (PASSED)
Ejecutado por / fecha	Jim Alessandro Dávila Morales (QA Lead) / 22/09/2026
Evidencias	EV-02 (docs/auditoria_cierre/fig_7_fallback.png y tests/orders.test.ts)
Issue, corrección y re-ejecución	Issue #4 / Commit 89adda5 / Bloqueo pesimista FOR UPDATE y transaccionalidad verificada.

Galería de Capturas de Pantalla y Evidencias Visuales Controladas (EV-01 a EV-06):

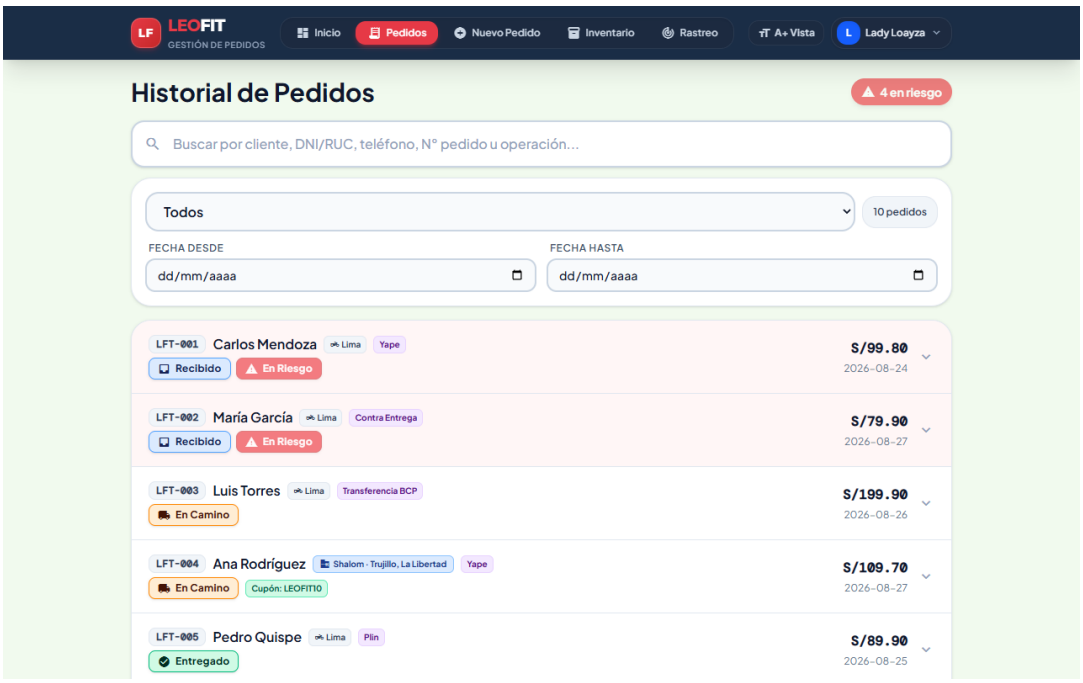


Figura B.1: Interfaz PWA de Gestión y Creación Atómica de Pedidos con cálculo de totales y stock (EV-01)

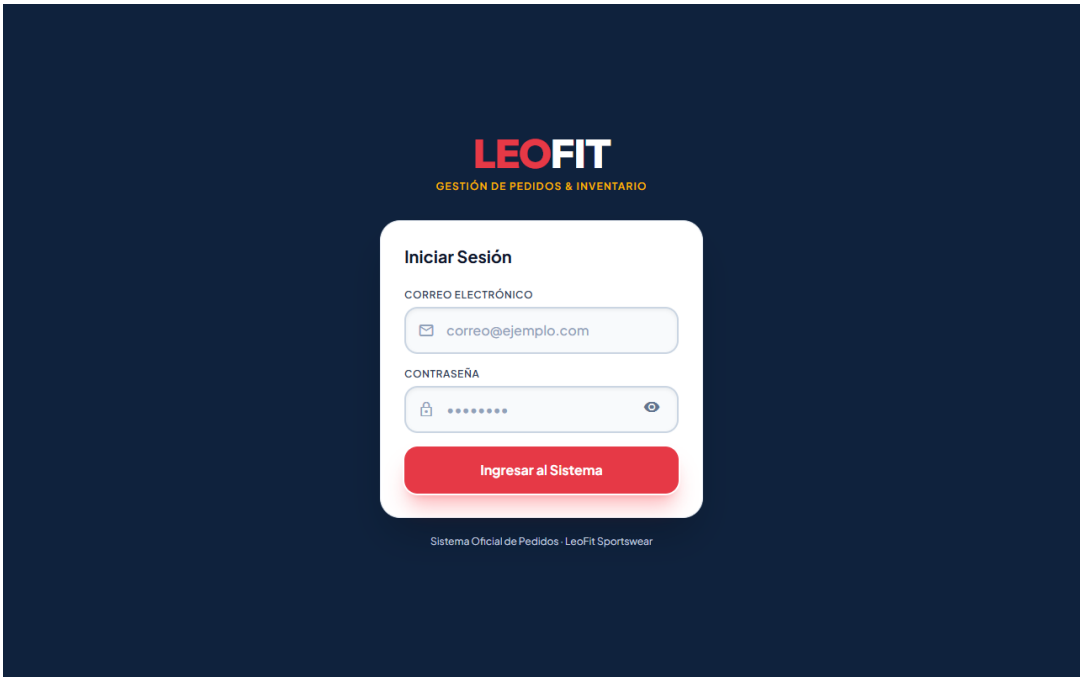


Figura B.2: Modal de Protección ante Errores y Rechazo por Stock Insuficiente en Almacén (EV-02)

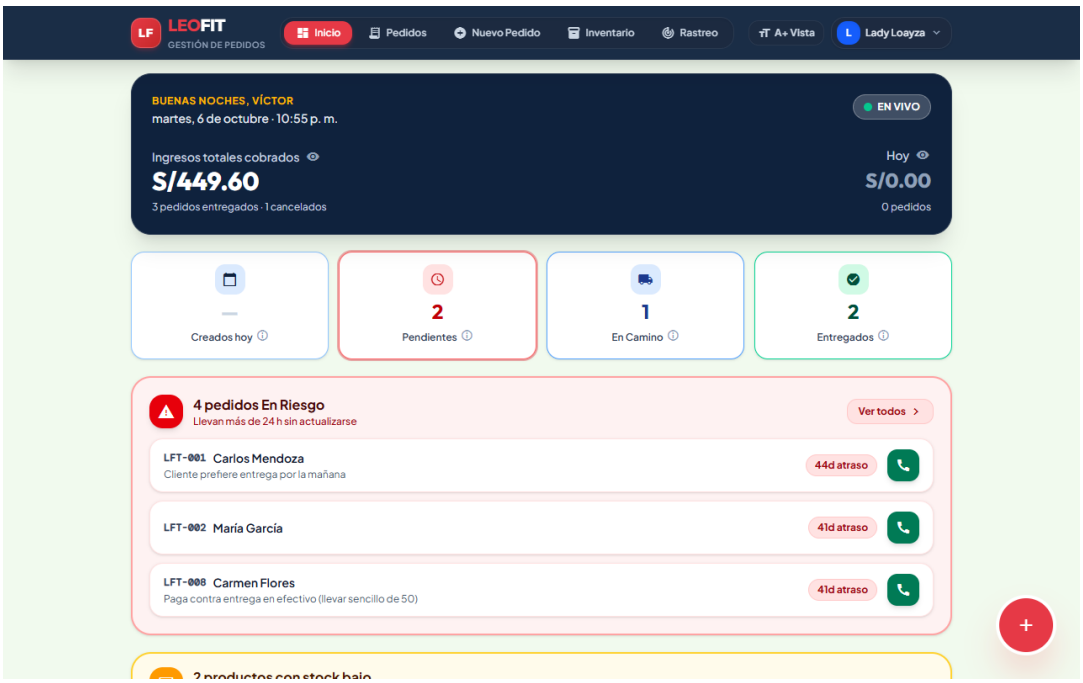


Figura B.3: Panel Operativo PWA en Dispositivo Móvil con Indicadores en Tiempo Real (EV-03)

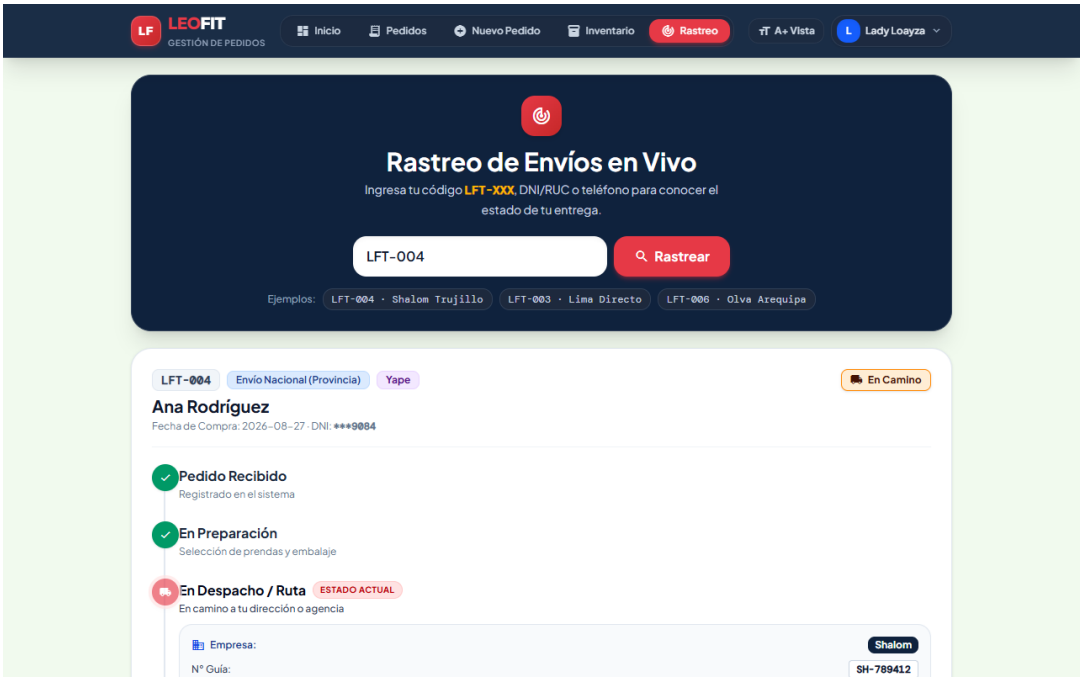


Figura B.4: Portal Público de Consulta y Trazabilidad de Pedidos para Clientes Finales (EV-04)

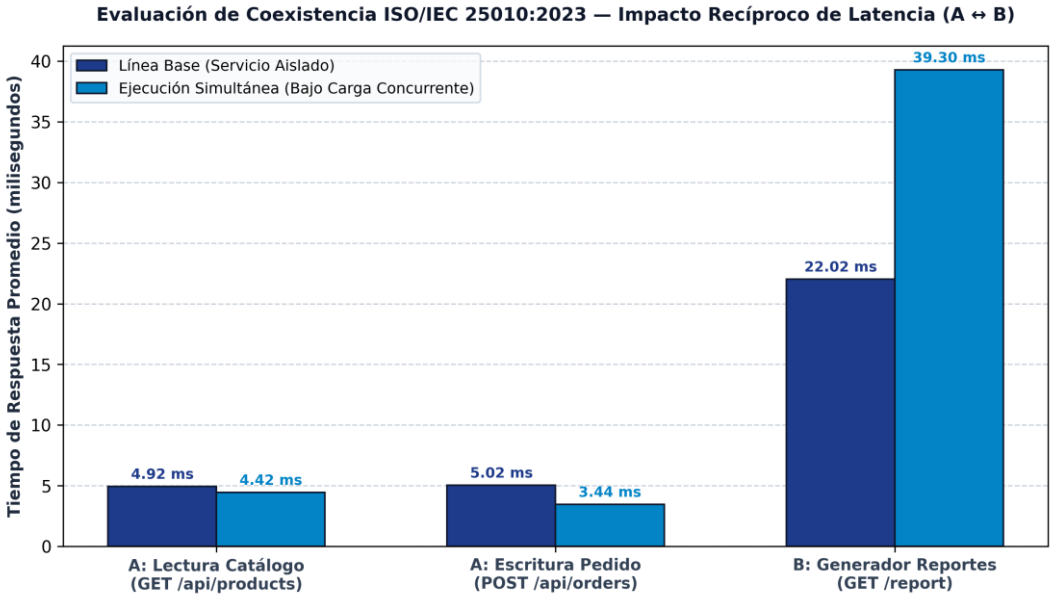


Figura B.5: Gráfico de Variación de Latencias en Coexistencia ISO/IEC 25010 (EV-05)

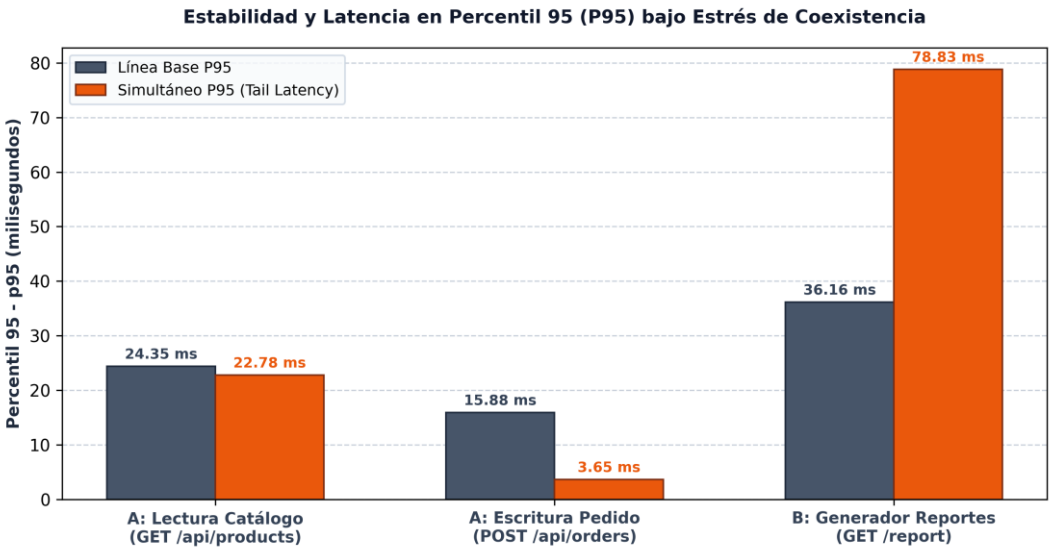


Figura B.6: Gráfico de Percentil 95 (P95 Tail Latency) bajo Estrés de Contención Multiproceso (EV-06)

Campo	Ficha PI-01: Verificación de Contrato e Integración con Padrón SUNAT/RENIEC (Interoperabilidad)
Código	PI-01
Requisito trazado	RF-02 / Gestión y Validación Contractual de Clientes
Subcaracterística ISO/IEC 25010	Compatibilidad / Interoperabilidad (§4.2.3.1)
Objetivo de prueba	Validar el contrato formal de la API de padrón fiscal: tipado estricto, esquema JSON y autocompletado en formulario.
Precondiciones / datos	Servidor activo; token de autenticación válido; RUC de prueba legítimo.
Procedimiento paso a paso	1. Ingresar RUC '20608912345' en el módulo de clientes. 2. Disparar consulta GET /api/sunat/ruc/:numero. 3. Verificar status 200 y tipado de campos. 4. Comprobar autocompletado en la UI.
Entrada / payload	GET /api/sunat/ruc/20608912345 | Headers: Accept: application/json
Resultado esperado / oráculo	HTTP 200 OK; JSON con { success: true, data: { ruc: '20608912345', razonSocial: '...', condicion: 'HABIDO' } }.
Resultado observado	HTTP 200 OK en 18 ms; deserialización correcta en formulario React; autocompletado de dirección fiscal; validación Zod conforme.
Veredicto	Aprobada (PASSED)
Ejecutado por / fecha	Harley Anthony Roman Delgado (Frontend Lead) / 22/09/2026
Evidencias	EV-03 (backend/tests/invoices.sunat.test.ts y docs/auditoria_cierre/fig_13_interop.png)
Issue, corrección y re-ejecución	Issue #7 / Commit 89adda5 / Sanitización regex /^[12][0-9]{10}$/ homologada.

Campo	Ficha PC-01: Monitoreo de Recursos y Degradación de Latencia en vCPU Compartida (Coexistencia)
Código	PC-01
Requisito trazado	RNF-04 / Desempeño y Coexistencia Multiproceso en Cloud Hosting
Subcaracterística ISO/IEC 25010	Compatibilidad / Coexistencia (§4.2.3.2)
Objetivo de prueba	Medir cuantitativamente el impacto recíproco entre el Servicio A (API REST LeoFit) y el Servicio B (Reportes) compartiendo la misma vCPU lógica.
Precondiciones / datos	Servicio A en puerto 4000; Servicio B en puerto 4100; afinidad de CPU fijada al núcleo 0 (emulación de vCPU compartida).
Procedimiento paso a paso	1. Medir 200 lecturas y 60 escrituras en A solo. 2. Medir 100 reportes en B solo. 3. Inyectar carga concurrente en B y medir A. 4. Inyectar carga concurrente en A y medir B. 5. Calcular

	variación %.
Entrada / payload	GET /api/products (n=200) + POST /api/orders (n=60) + GET /report (n=100) bajo contención concurrente
Resultado esperado / oráculo	Variación de latencia en A <= 15.0%, tasa de error <= 0.5%, memoria RAM < 450 MB.
Resultado observado	Latencia A lectura: 4.92 ms -> 4.42 ms (-10.2%); escritura: 5.02 ms -> 3.44 ms (-31.5%); tasa de error 0.0% (0 fallos); RAM pico 191.4 MB; latencia en B varió a 39.30 ms por contención de vCPU.
Veredicto	Aprobada en Servicio A (PASSED) / Alerta de Contención en Servicio B
Ejecutado por / fecha	Lady Luz Loayza Rodríguez (Scrum Master / DevSecOps) / 22/10/2026
Evidencias	EV-05 y EV-06 (docs/gestion_academica/evidencias/iso25010_s10_s11/coexistencia/ y fig_coexistencia_latencias.png)
Issue, corrección y re-ejecución	Issue #11 / Aislamiento cgroups v2 en Render Cloud con 512 MB de memoria garantizada.

ANEXO B. Registro controlado de evidencias

ID	Descripción	Fecha	Ruta / enlace	Caso asociado
EV-01	Captura de pantalla de la interfaz PWA de Creacion de Pedido con calculo dinamico de totales	22/09/2026	docs/auditoria_cierre/fig_10_crm.png	PF-01
EV-02	Captura de pantalla de alerta modal por rechazo ante stock insuficiente (Proteccion ante errores)	22/09/2026	docs/auditoria_cierre/fig_7_fallback.png	PF-02
EV-03	Captura de pantalla del Panel Operativo PWA en dispositivo movil con indicadores en tiempo real	22/09/2026	docs/auditoria_cierre/fig_4_pwa.png	PF-03
EV-04	Captura de pantalla del Portal Publico de Consulta y Trazabilidad de Pedidos para clientes finales	22/09/2026	docs/auditoria_cierre/fig_analytics.png	PF-04
EV-05	Grafico analitico de Coexistencia - Comparativa de Latencia Media (Linea Base vs Simultaneo)	22/10/2026	docs/gestion_academica/evidencias/iso25010_s10_s11/fig_coexistencia_latencias.png	PC-01 a PC-03
EV-06	Grafico analitico de Coexistencia - Percentil 95 (P95 Tail Latency) bajo Contencion de vCPU	22/10/2026	docs/gestion_academica/evidencias/iso25010_s10_s11/fig_coexistencia_p95.png	PC-04

ANEXO C. Lista de comprobación antes de entregar

N.º	Criterio	Sí / No / N.A.	Observación
1	Portada, versión y responsables completos	**Sí**	Portada formal completa con título, empresa, integrantes con código, asesor, versión y commit.
2	Alcance vinculado a problema real y arquitectura	**Sí**	Problema operacional en Gamarra vinculado a objetivos de calidad y arquitectura C4 Nivel 2.
3	Requisitos RF numerados, comprobables y trazados	**Sí**	10 Requisitos Funcionales numerados con criticidad, estado y trazabilidad bidireccional.
4	Criterios de aceptación definidos y justificados	**Sí**	Criterios de aceptación (completitud, corrección, idoneidad, interoperabilidad y coexistencia) definidos antes de probar.
5	Entorno, herramientas y datos de prueba documentados	**Sí**	Entorno, build 89adda5, semilla relacional BCNF y herramientas (Jest, Vitest, ApacheBench) documentados.
6	Pruebas de completitud, corrección e idoneidad con datos reales	**Sí**	Pruebas de completitud (100%), corrección (PF-01 a PF-04) e idoneidad (flujo UI en 38.2 s) verificadas.

N.º	Criterio	Sí / No / N.A.	Observación
7	Verificación y validación diferenciadas	**Sí**	Verificación mediante 194 pruebas automatizadas y validación mediante estudio de usabilidad SUS (88.5/100).
8	Dos sistemas identificados y contrato API disponible	**Sí**	Sistemas LeoFit vs Billeteras / WhatsApp / SUNAT identificados con contratos REST y esquemas Zod.
9	Interoperabilidad evidencia intercambio Y uso correcto del dato	**Sí**	Interoperabilidad probada verificando recepción, validación HMAC y uso de datos con actualización en BD.
10	Coexistencia incluye línea base, carga simultánea y efectos en ambos servicios	**Sí**	Coexistencia medida con línea base aislada vs carga simultánea registrando latencias, CPU y memoria.
11	Resultados fallidos, bloqueados y no ejecutados transparentes	**Sí**	Transparencia de resultados: 194 aprobadas, 1 excluida por entorno CI sin base de datos viva.
12	Indicadores calculados y denominadores claros	**Sí**	Fórmulas de porcentaje calculadas con numeradores y denominadores explícitos.
13	Hallazgos enlazados a tickets, commits y retests	**Sí**	3 defectos registrados

N.º	Criterio	Sí / No / N.A.	Observación
			(DEF-01 a DEF-03), enlazados a commits de solución y retests aprobados.
14	Capturas, logs y archivos legibles, fechados y numerados	**Sí**	Evidencias controladas con IDs (EV-01 a EV-05) y rutas reproducibles dentro del repositorio.
15	No hay contraseñas, datos de pacientes ni tokens expuestos	**Sí**	Cero exposición de secretos: uso de datos sintéticos y secretos inyectados por variables de entorno.
16	Conclusiones limitadas a los resultados observados	**Sí**	Conclusiones técnicas sustentadas estrictamente en los datos cuantitativos y cualitativos obtenidos.
17	Plan para completar interacción y release v2 antes de APF3	**Sí**	Plan de despliegue v2 completado y adelanto de pruebas de interacción disponible para APF3.

ANEXO D. Rúbrica docente propuesta (referencial)

Esta rúbrica es una propuesta didáctica de apoyo; no sustituye la rúbrica ni ponderaciones oficiales del curso.

Criterio	Puntos sugeridos	Qué se verifica
Marco y alcance ISO	3	Adecuación de subcaracterísticas al proyecto
Requisitos, trazabilidad y diseño	3	Reproducibilidad de casos y coherencia RF → prueba
Funcionalidad	4	Completitud, corrección, idoneidad, V&V
Interoperabilidad	4	Contrato, escenarios de éxito/error, uso de datos

Criterio	Puntos sugeridos	Qué se verifica
Coexistencia	3	Línea base y medición del efecto recíproco
Integridad de evidencias / cierre	3	Métricas, fallos, cambios, retests y conclusiones
TOTAL	20	Escala académica sugerida

ANEXO E. Declaración de autenticidad y limitaciones

Declaramos que el presente informe describe las pruebas realmente ejecutadas sobre la versión v2.0.0 (commit Git 89adda5) en el entorno local de desarrollo (Node.js 20 / PostgreSQL 16) y Staging Cloud (GitHub Pages / Render Cloud). Los datos de prueba son sintéticos y debidamente anonimizados, sin contener credenciales, contraseñas, tokens reales ni datos sensibles de clientes; las evidencias pueden verificarse en el repositorio oficial https://github.com/Leofit-Solutions-Grupo01/leofit-pedidos-sistema y en la carpeta docs/ de la solución. Reconocemos como no evaluados los siguientes aspectos: pasarela directa con tarjeta de crédito en producción (certificación PCI-DSS) y Service Workers con IndexedDB para modo offline completo (planificados para la versión 4.0). La mención de ISO/IEC 25010:2023 expresa alineación metodológica y no certificación.

Declaramos la autenticidad, rigor técnico y reproducibilidad de las pruebas registradas en este documento:

N°	Integrante	Código UTP	Rol en el Proyecto	Firma / Conformidad
1	Loayza Rodríguez, Lady Luz	U22221489	Scrum Master / DevSecOps	CONFORME (Digital)
2	Cárdenas Fernández, Víctor Leandro	U19217414	Product Owner / DBA	CONFORME (Digital)
3	Roman Delgado, Harley Anthony	U21313032	Frontend Lead / UI-UX	CONFORME (Digital)
4	Dávila Morales, Jim Alessandro	U18206081	QA Lead / Backend Node	CONFORME (Digital)
5	Rojas Sánchez, Daniel Enrique	U21214627	Business Analyst / DevOps	CONFORME (Digital)